Project Report

Remote Area Resource Management System.

Submitted By:

Aliyan Ali AKbar

[24-CS-063]

[SECTION C]

SUB:

COAL

Submitted to:
SIR KHALID

Department of Computer Science,

HITEC University, Taxila

Remote Area Resource Management System

CCP Project Report

---

1. Introduction

The Remote Area Resource Management System is a menu-driven application developed using 8086 Assembly Language on the EMU8086 platform. The main purpose of this project is to manage and organize resource distribution data for families living in remote areas. The system efficiently stores, updates, deletes, displays, sorts, and analyzes records related to essential resources such as water, flour, and pulses.

This project demonstrates practical usage of assembly language concepts, including memory management, array handling, procedure modularization, stack operations, and DOS interrupt services.

---

2. Objectives of the Project

The primary objectives of this CCP project are:

- To understand low-level programming using 8086 Assembly Language

- To implement a record management system without using high-level constructs
- To practice the use of arrays, loops, procedures, and stack operations
- To manage real-world data such as family details and resource consumption
- To perform sorting and statistical analysis using assembly language

---

3. Scope of the Project

This system is designed for small-scale data management with a maximum capacity of 50 records. It is suitable for academic and learning purposes and can be extended in the future for larger datasets or enhanced functionality.

---

4. Tools and Technologies Used

- Programming Language: 8086 Assembly Language
- Assembler/Emulator: EMU8086
- Operating Environment: DOS-based Interrupts (INT 21H)
- Memory Model: SMALL model

---

5. System Features

The Remote Area Resource Management System provides the following features:

1. Add New Record

2. Update Existing Record
3. Delete Record
4. Display All Records
5. Sort Records by Family Members
6. Sort Records by Water Consumption
7. Sort Records by Flour Consumption
8. Sort Records by Pulses Consumption
9. Display Total Statistics
10. Exit Program

---

6. Data Structure Design

The system uses parallel arrays to store data efficiently:

- SR_NO: Stores serial numbers of families
- NAMES: Stores family names (20 characters per record)
- FAMILY_MEMBERS: Number of members in each family
- WATER_CONSUME: Water consumption in liters
- FLOUR_CONSUME: Flour consumption in kilograms
- PULSES_CONSUME: Pulses consumption in kilograms

Each record is accessed using index registers (BX, SI, DI) as per 8086 architecture limitations.

---

7. Functional Description

7.1 Add Record

- Checks if the database is full
- Takes user input for all fields

- Ensures no duplicate serial number exists
- Stores data in respective arrays

7.2 Update Record

- Searches record using serial number
- Allows updating name and resource values
- Displays error if record is not found

7.3 Delete Record

- Finds the record using serial number
- Shifts all subsequent records to maintain continuity
- Decreases record count

7.4 Display Records

- Displays all stored records in a tabular format
- Uses formatted output for clarity

7.5 Sorting Records

- Implements Bubble Sort algorithm
- Sorting can be performed based on:
    - Family members
    - Water consumption
    - Flour consumption
    - Pulses consumption

7.6 Display Statistics

- Calculates total:
    - Family members
    - Water consumption
    - Flour consumption

    - Pulses consumption
- Uses stack-based accumulation

---

8. Algorithms Used

- Linear Search: For finding and validating records
- Bubble Sort: For sorting records based on selected criteria
- Stack-Based Arithmetic: For calculating totals

---

9. Error Handling

The system handles common errors effectively:

- Duplicate record detection
- Database full condition
- Empty database condition
- Record not found error

Appropriate error messages are displayed for user guidance.

---

10. Limitations

- Maximum of 50 records only
- No persistent storage (data lost after program exit)
- Text-based interface only
- Manual input validation

---

11. Future Enhancements

- File handling for permanent data storage
- Enhanced input validation
- Support for larger datasets
- Improved user interface
- Graphical visualization of statistics

---

12. Conclusion

This project successfully demonstrates how a complete resource management system can be implemented using 8086 Assembly Language. It strengthened understanding of low-level programming concepts such as memory addressing, procedure handling, and DOS interrupts.